Learning Objectives

After completing this worksheet, you should be able to:
- distinguish between valid, uninitialized, and null pointers and explain why dereferencing an invalid pointer results in undefined behavior;
- relate the initialization of pointer variables to their storage location (in particular, global variables in the BSS/data segment versus local variables on the stack);
- use `nullptr` to represent an invalid pointer deliberately and to mark the end of a linked data structure;
- use `malloc` to dynamically allocate memory for objects, in particular for struct objects;
- use `sizeof` to determine the amount of memory required for an allocation;
- combine *structs*, *pointers*, and *dynamic memory* to create and traverse a simple singly linked list;
- understand the basic responsibility associated with dynamically allocated memory, in particular that allocated memory eventually has to be released with `free`.

Step 1 — Invalid Pointers

In the following program, uninitialized pointers are dereferenced:

```
 1 @ <stdio.hdr>
 2 @ <stdlib.hdr>
 3
 4 global q: ->int;
 5
 6 fn main()
 7 {
 8     local p: ->int;
 9
10     *p = 4;
11     *q = 5;
12 }
```

The program compiles successfully, but when it is executed, it terminates with a *segmentation fault* (which is usually the operating system's way of giving you the middle finger)[1].

Tasks

1. Print the values of the pointer variables `p` and `q` before they are dereferenced. Compare their values and explain why they are different.

   Hint: Recall the discussion of the memory layout in the lecture video. Where are the global variable `q` and the local variable `p` stored? What did we learn about the initial contents of these memory regions?

2. Comment out the assignment in line 11. The program may now run without crashing. Explain why it is actually bad news if the program does not crash.

Step 2 — Null Pointers

A pointer is valid if it points to an *allocated memory region*. Such memory regions include those occupied by global variables, local variables (as long as they are still within the valid part of the stack), and memory that has been allocated dynamically (for example, using `malloc`) and has not yet been released.

To ensure well-defined behavior, local pointer variables should be initialized either with a valid address or with the constant nullptr. In C, C++, and ABC, `nullptr` represents an address that is guaranteed to be invalid: dereferencing a pointer whose value is `nullptr` will cause the program to crash.

Task: In the program from the previous step, initialize the local variable `p` with `nullptr`. Confirm that the program now crashes when `p` is dereferenced.
Remark: Whether `nullptr` is actually represented by the value `0` is platform-dependent.

---

[1] More precisely, and less vulgar, the program attempts an invalid memory access, which is detected by the hardware and handled by the operating system. In a larger program, finding the actual cause can be notoriously difficult: the pointer may have become invalid far away from the place where it is eventually dereferenced. Even worse, undefined behavior involving invalid pointers may not fail deterministically—the same test may pass on one run and crash on the next.

© Michael C. Lehn | CC BY-SA 4.0

Step 3 — A Pattern for Using `malloc`

Memory can be allocated dynamically by calling `malloc`. However, an allocation can fail[2]. In that case, `malloc` returns `nullptr`.

```
1 @ <stdio.hdr>
2 @ <stdlib.hdr>
3
4 fn main()
5 {
6     local p: ->int = malloc(4);
7     if (p == nullptr) {
8         printf("out of memory\n");
9         exit(1);
10    }
11 }
```

The return value of `malloc` should therefore always be checked. If the allocation fails, the program should terminate with a nonzero exit code. The following program prints an error message and terminates with exit code `1` in this case (the function `exit` is declared in `<stdlib.hdr>`):

Tasks:

1. Compile and run the program above. Use the shell command `echo $?` to inspect its exit code.
2. By convention, an exit code of `0` indicates that a program terminated successfully. For testing purposes, modify the program so that it returns exit code `42` after a successful allocation.
3. In line 7, `p == nullptr` is used to test whether `p` is a valid pointer. However, it is not necessary to compare `p` explicitly with `nullptr`. When a pointer is used in a logical expression, `nullptr` is interpreted as `false`, while any other pointer value is interpreted as `true`.

   Modify the condition in line 7 so that nullptr is no longer used explicitly.
4. The argument passed to `malloc` specifies the size of the memory region to be allocated, in **bytes**. In the example above, the value 4 was used because `int` has a size of four bytes on many platforms. A portable—and therefore better—solution is to let the compiler determine the actual size using the `sizeof` operator:

   ```
   6     local p: ->int = malloc(sizeof(int));
   ```

   However, this introduces another potential problem: if the type of `p` is changed later, it is easy to forget to change the type used with `sizeof`:

   ```
   6     local p: ->u64 = malloc(sizeof(int));
   ```

   Solve this problem. Use `sizeof` so that the amount of memory passed to `malloc` always corresponds to the size of the object pointed to by `p`, **without explicitly specifying its type**.
5. Consider the following program, in which p is declared as a global pointer variable:

   ```
   1 @ <stdio.hdr>
   2 @ <stdlib.hdr>
   3
   4 global p: ->int = malloc(sizeof(*p));
   5
   6 fn main()
   7 {
   8     if (p == nullptr) {
   9         printf("out of memory\n");
   10        exit(1);
   11    }
   12 }
   ```

   The compiler refuses to compile this program. Initial values of global variables must be known at compile time. Explain why this is not the case here.

   Then modify the program so that p is assigned the result of malloc at runtime.

---

[2] A common reason is that the requested amount of memory cannot be provided. More generally, an allocation may fail because of system or process limits or because sufficient virtual address space is not available. Also note that on modern systems, a successful call to `malloc` does not necessarily mean that the corresponding amount of physical memory has already been made available; virtual memory and mechanisms such as memory overcommit can make the situation more subtle.

© Michael C. Lehn | CC BY-SA 4.0

Step 4 — Memory Leaks and Using `free`

Memory allocated with `malloc` remains allocated until the program terminates or the memory is released using `free`. To avoid memory leaks, every successful call to `malloc` should eventually have a corresponding call to `free`.

These important rules should be followed:

(i) Every successful allocation should eventually be released.
(ii) Pass to `free` only the address returned by `malloc` (or `nullptr`), never an address inside the allocated block.
(iii) Each successfully allocated block must be released at most once.

The following program compiles successfully, but violates rules (ii) and (iii):

```
1 @ <stdio.hdr>
2 @ <stdlib.hdr>
3
4
5 fn main()
6 {
7     local p: ->int = malloc(sizeof(*p));
8     if (p == nullptr) {
9         printf("out of memory\n");
10        exit(1);
11    }
12
13    ++p;
14    free(p);
15    free(p);
16 }
```

Whether the program actually crashes is not guaranteed. Its behavior is **undefined**.

Test the program in the following cases:

1. Compile and run the program as shown above.
2. Comment out line 15.
3. Restore line 15, but comment out line 13 instead.

Compare your results with those of others. In particular, compare them with results obtained on a different platform (for example, a different operating system).

Remark

Two things may seem surprising:

1. `malloc` can be used to allocate memory for objects of different data types.
2. `free` only needs the starting address of an allocated memory region—the size of the region does not have to be specified.

Here is a brief explanation of both:

1. The return type of `malloc` is `->void`, commonly called a *void pointer*. This does not mean an invalid pointer. Rather, it is a pointer for which the type of the object it points to is unspecified. In the compiler's bookkeeping, you can think of this type as being marked as unknown.

   A void pointer can be converted to another pointer type. It is the programmer's responsibility to ensure that this conversion makes sense—in particular, that the pointer is subsequently used only to access memory in a valid way.
2. `malloc` and `free` can be implemented in different ways. One possibility is that `malloc` allocates slightly more memory than requested and uses the additional memory to store bookkeeping information, such as the size of the allocated block. `free` can then use this information.

   For example, if `malloc` successfully allocates k bytes and returns the address stored in `p`, the memory might conceptually look like this:



© Michael C. Lehn | CC BY-SA 4.0

Step 5 — A First Singly Linked List

Before looking at the technical details, compile and run the following program:

```
1  @ <stdio.hdr>
2
3  struct Node
4  {
5      val: array[8] of char;
6      next: ->Node;
7  };
8
9  global a, b, c: Node =
10     {
11         {"world", &b},
12         {"hello,", &c},
13         {"!"},
14     };
15
16 fn main()
17 {
18     for (local p: ->Node = &a; p; p = p->next) {
19         printf("%s", p->val);
20     }
21     printf("\n");
22 }
```

At first, it may seem surprising that the compiler accepts this code: in lines 3–7, the type `Node` is defined, but `Node` already occurs within its own definition in line 6.
A simplified explanation of why this works is:

(a) From line 3 onward, the compiler knows that a type named `Node` is being defined. Its size, however, is not known until the end of the definition (after line 7).
(b) To determine the size of the struct, the compiler must know the size of each member. The member `next` in line 6 is a pointer to `Node`. The size of a pointer does not depend on the size of the object it points to.

Remark: This example deliberately builds a linked list without dynamic memory allocation. The three nodes already exist as global variables. This is somewhat unusual in practice, but it allows us to separate two ideas: first, how nodes can be connected by pointers to form a linked list; and second, how such nodes can later be created dynamically using malloc.

Tasks:

1. In line 9, the variables `a`, `b`, and `c` are defined and initialized. Sketch the corresponding memory regions and their contents.
2. Explain which values the local variable `p` takes while the `for` loop in lines 18–20 is executed.
3. Modify line 18 and the initialization of the respective `next` members in lines 11–13 so that the program prints `hello, world!`

© Michael C. Lehn | CC BY-SA 4.0

Step 6 — Dynamically Allocated Linked Lists

In the previous step, we constructed a linked list from nodes that already existed as global variables. We will now create the nodes dynamically using `malloc`.

Consider the following program `list.abc`:

```
1 @ <stdio.hdr>
2 @ <stdlib.hdr>
3
4 struct Node
5 {
6     val: int;
7     next: ->Node;
8 };
9
10 fn main()
11 {
12     local list: ->Node = nullptr;
13
14     // prepend new node to list
15     local n1: ->Node = malloc(sizeof(*n1));
16     n1->val = 42;
17     n1->next = list;
18     list = n1;
19
20     // prepend new node to list
21     local n2: ->Node = malloc(sizeof(*n2));
22     n2->val = 12;
23     n2->next = list;
24     list = n2;
25
26     printf("list->val = %d\n", list->val);
27     printf("list->next->val = %d\n", list->next->val);
28 }
```

Remark: To keep our initial focus on the linked-list operations, the program above does not check whether malloc returned nullptr. In production code, this check must not be omitted. You already know the appropriate pattern from Step 3—and we will add it below.

Tasks:

1. Before analyzing the program in detail, type it in, compile it, and run it.
2. Now trace what happens in `main` statement by statement. In particular, describe how the relevant memory regions change after each statement.
   Start with

   ```
   12     local list: -> Node = nullptr;
   ```

   and then trace the creation of the first node:

   ```
   14     // prepend new node to list
   15     local n1: -> Node = malloc(sizeof(*n1));
   16     n1->val = 42;
   17     n1->next = list;
   18     list = n1;
   ```

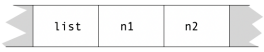
   Draw the corresponding memory regions and pointers after each statement. Remember that list and `n1` are local variables, while the `Node` created by `malloc` lives in dynamically allocated memory.
   Then do the same for the second node (`n2`).

© Michael C. Lehn | CC BY-SA 4.0

Hint: Visualizing What Happens

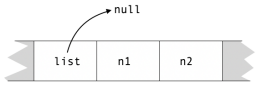
When working with pointers and dynamically allocated data structures, it is often helpful to draw the relevant memory regions and the pointers between them. There is no single correct way to do this. The art is to leave out enough technical detail to make the essential idea visible, while still showing enough detail for the picture not to become misleading. A useful visualization can also be refined later. If a particular technical detail becomes important, we can add it to the picture without having to abandon the simpler model.

In `main`, the three local variables `list`, `n1`, and `n2` are defined. For our visualization, we represent them by three distinct memory regions on the stack. We can think of the required stack space as being reserved when the function is entered, as part of setting up its stack frame. Initially, the contents of these memory regions are unspecified:

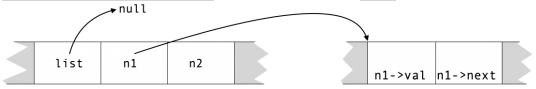


This picture deliberately leaves out some details. For example, an optimizing compiler may keep the value of a local variable in a CPU register instead of repeatedly storing and loading it from its stack location. For what we want to understand here, however, the simpler picture above is sufficient.

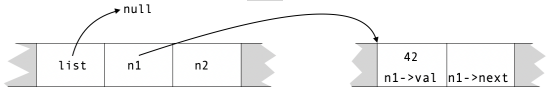
In **line 12**, `list` is initialized at runtime with the null pointer. We can visualize this effect as follows:



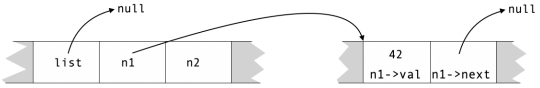
In **line 15**, `malloc` allocates a memory block of size `sizeof(*n1)` (which is equivalent to `sizeof(Node)` here), and `n1` is initialized with the address of the allocated memory block:



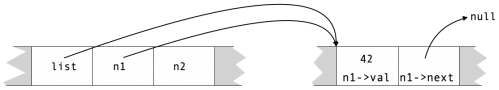
This memory block will be used as a new node of the list. In **line 16**, its member `val` is initialized with the value `42`:



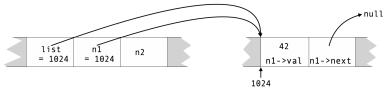
In **line 17**, the member `next` of the node is initialized with the current value of `list`:



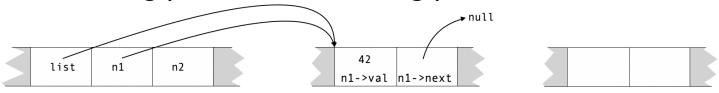
In **line 18**, list is set to the address of the node:



It can also be helpful to use concrete (but made-up) addresses for memory regions. This makes it explicit that a pointer variable really just stores an address. For example, suppose the block allocated by `malloc` has address `1024`, and `nullptr` is represented by the value `0` on our architecture. Then, after **line 18**, we could also describe the memory regions as follows:



We will later say that **line 12** created an empty list, and that **lines 15–18** prepended a new node with value **42** to the list. However, the idea of prepending a new node to a list becomes much clearer when you now describe the effect of **lines 21–24** in the same way. You can use the following picture as a starting point and extend it step by step:



From your final memory diagram, it becomes apparent that a list with two nodes has been created: the first node contains value `12`, and the second node contains value `42`. A node is identified as the last node of the list by having `nullptr` as the value of its next member.

© Michael C. Lehn | CC BY-SA 4.0

3. Replace the two printf statements by

```
printf("list:\n");
for (local n: ->Node = list; n != nullptr; n = n->next) {
    printf("n->val = %d\n", n->val);
}
```

   (a) Confirm that the program essentially produces the same output as before.
   (b) Explain which values `n` takes while the loop is executed and why the loop eventually terminates.
   (c) Remove the code that creates and prepends `n1`. Confirm that the list now contains only the node with value `12`.
   (d) Also remove the code that creates and prepends `n2`. The list is now **empty**. Confirm that the loop does not attempt to access invalid memory and simply produces no output for the list elements.
   (e) In C, C++, and ABC, a pointer used as a condition does not have to be compared explicitly with `nullptr`. Rewrite the loop condition accordingly.

4. Finally, replace the explicit creation of `n1` and `n2` by a loop:

```
for (local i: int = 1; i <= 3; ++i) {
    local n: ->Node = malloc(sizeof(*n));
    n->val = i;
    n->next = list;
    list = n;
}
```

   Again, sketch the relevant memory regions as the program executes and explain how each new node is prepended to the list.
   We have deliberately ignored one important detail so far. As pointed out in the remark above, it is now time to check whether `malloc` actually succeeded. Add the appropriate check to the loop. If the allocation fails, print an error message and terminate the program with a nonzero exit code.
   Finally, modify the program so that it creates a list containing `42` nodes[3].

5. Code for printing the list can be moved into a function:

```
fn printList(list: ->Node)
{
    for (; list; list = list->next) {
        printf("val = %d\n", list->val);
    }
}
```

   Use this function in `main` to print the list. The local variable `list` is modified inside function `printList`. Explain to your neighbor (or to me in the exam) why the value of `list` in `main` is nevertheless unchanged after the call to `printList`.

6. There is another way to print the list. Consider the following recursive function:

```
fn printListRecursive(list: ->Node)
{
    if (list) {
        printf("val = %d\n", list->val);
        printListRecursive(list->next);
    }
}
```

   First, make sure that you understand how this function works. In particular, think about what happens for the last node of the list and why the recursion eventually terminates.

   Call `printListRecursive` from `main` and verify that it prints the same list as `printList`.

   Now write a recursive function `printListReverse` that prints the elements of the list in reverse order.

[3] Asking you to create exactly `42` nodes is my way of telling you: Your code is now general enough to generate `n` nodes. ;-)

© Michael C. Lehn | CC BY-SA 4.0

Chillout — Putting It All Together

We have now seen the essential ingredients of a dynamically allocated linked list. In a larger program, however, we would not want to repeat the pointer manipulations throughout our code. Instead, operations on the list should be moved into functions.
The following program shows what this can look like. There are no tasks on this page. Just read the code and see how much of it you already understand.

```
@ <stdio.hdr>
@ <stdlib.hdr>

struct Node
{
    val: int;
    next: ->Node;
};

fn prependToList(list: ->->Node, val: int)
{
    local n: ->Node = malloc(sizeof(*n));
    assert(n);
    n->val = val;
    n->next = *list;
    *list = n;
}

fn printList(list: ->Node)
{
    for (; list; list = list->next) {
        printf("val = %d\n", list->val);
    }
}

fn freeList(list: ->Node)
{
    if (list) {
        freeList(list->next);
        free(list);
    }
}

fn main()
{
    local list: ->Node = nullptr;

    for (local i: int = 1; i <= 42; ++i) {
        prependToList(&list, i);
    }

    printf("list:\n");
    printList(list);

    freeList(list);
}
```

Two details may still look somewhat mysterious.

In `prependToList`, the parameter `list` is a pointer to a pointer. This allows the function to change the value of `list` in `main`. Later, we will see in detail how this implements *call by reference* using *call by value*.
In `freeList`, the order of operations is important. The remainder of the list is freed before the current node is freed. Once `free(list)` has been called, accessing `list->next` would no longer be valid. Notice how closely this resembles the recursive traversal from the previous task.

For later: When preparing for the exam, come back to this program and check whether you can explain every line. A later worksheet will develop the two less obvious parts—modifying the head of a list and correctly freeing an entire list—in detail.

© Michael C. Lehn | CC BY-SA 4.0